WEB APPLICATION / IMPLEMENTATION / ARTIFACTS

StudySprint

Project and Artifact Report

A browser-based study planner with the SprintBot quick planning assistant.

Project owner: wrightiv-dotcom **Report date:** 29 October 2026

Live app: https://wrightiv-dotcom.github.io/studysprint/
GitHub: https://github.com/wrightiv-dotcom/studysprint

1 Purpose and implemented scope

StudySprint helps students turn assignment requirements, deadlines, and available study time into smaller milestones and realistic sessions. Instead of maintaining only a deadline list, a student can review how the work will be divided and whether it fits before submission.

The application includes an Express JavaScript framework backend and an HTML, CSS, and JavaScript frontend. A static local-planning edition is live on GitHub Pages; the framework edition is prepared for Render. Both include assignment entry, editable milestone estimates, daily availability, schedule validation, approval, completion, and replanning. The design uses an animated night sky, translucent navy panels, and light text and icons.

SprintBot offers separate local and AI modes. Local mode uses rules and templates. The new server-side AI agent uses the OpenAI Responses API and three function tools to inspect context, check a proposed plan, and revise it. **Live AI activation still requires a Render connection and a privately configured API key.** The interface does not describe local templates as model-generated output.

1.1 Typical user workflow

1. Set available study windows for each day of the week.
2. Enter an assignment title, course, deadline, requirements, and estimated effort, or open **Plan with SprintBot** for a quick proposal.
3. Review the milestone titles and time estimates. Check the proposal against the actual assignment rubric.
4. Check schedule feasibility. If insufficient time is available, adjust availability or estimates and review again.
5. Approve the plan to place its sessions in the study schedule.
6. Mark sessions complete and replan unfinished work when circumstances change.

1.2 Success criteria

A usable proposal must fit within available windows, avoid other approved sessions, and finish before the assignment deadline. The application must explain an infeasible workload and require approval before scheduling a revised plan. These checks concern time allocation; they do not establish the academic quality of a student's eventual submission.

2 Architecture and project artifacts

2.1 Client-side architecture

The browser loads the interface and stores assignments in `localStorage` under `studysprint-v1`. Express serves only the public `dist/` folder and exposes health, configuration, and planning endpoints. The AI endpoint orchestrates model tool calls; local mode remains in the browser. There is no account database. The model key is read only from the server environment and never sent to the frontend.

Assignment + availability inputs

↓

SprintBot proposal or editable milestone draft

↓

Schedule checks ⟶ review and explicit approval

↓

Browser storage + dashboard + completion / replanning

2.2 Artifact inventory

The repository separates editable website source from the folder published by GitHub Pages. Both contain the required web assets.

Artifact	Purpose
`dist/index.html`	Dashboard, assignment forms, dialogs, navigation, and script loading.
`dist/style.css`	Responsive layout, base styling, and night sky theme.
`dist/script.js`	Application state, scheduling, approval, progress, and replanning.
`dist/sprint-planner.js`	Pure planning logic: assignment categories, task lists, and time distribution.
`dist/sprintbot.js`	Robot interface, draft preview, schedule handoff, and optional ChatGPT prompt.
`dist/sprintbot.css`	Robot launcher, assistant dialog, and milestone preview styles.
`dist/assets/clouds-night-sky.gif`	Animated background asset included with the website.
`docs/`	Published copy of the site, including `.nojekyll`.
`scripts/prepare-pages.cjs`	Copies website files from `dist/` into `docs/`.
`tests/`	Six planner tests, six agent tests, and four API tests.
`package.json`, `README.md`	Test/publishing commands and project instructions.
`report/studysprint-report.tex`	Editable LaTeX source for this report.
`output/pdf/studysprint-report.pdf`	Compiled report artifact.

2.3 Storage and privacy boundary

Browser storage is local to the visitor's browser and site origin. It is not a backup or cross-device synchronization service. Clearing browser data can remove saved assignments. The repository and rendered website are public, but a visitor's local assignments are not committed to GitHub or shared with other visitors by the application.

3 Planning behavior and SprintBot

3.1 Milestone generation

SprintBot accepts a new assignment or an existing assignment, requirements, a deadline, an assignment category, and a total estimate. Automatic category selection uses keywords for writing, coding, exams, and presentations; unmatched work uses a general template. This is a heuristic and may be overridden by the student.

If the requirements contain between two and eight nonempty task lines of sufficient length, those lines become the proposed milestones. Otherwise, the selected category supplies four starter milestones. The planner distributes the total effort in whole minutes across the proposed steps. It validates required text, a deadline that has not passed, and an estimate between 0.5 and 200 hours. All suggested titles and estimates remain reviewable.

3.2 Scheduling and approval

The scheduler examines daily study windows from the current date through the deadline, subject to a 730-day search limit. Today's sessions start no earlier than the next applicable quarter-hour. Other approved, unfinished sessions are treated as occupied time. Proposed sessions last up to 60 minutes and may be shorter at a window boundary or to finish a milestone.

If remaining work cannot fit, the scheduler returns a shortfall in minutes. Approval is disabled for invalid estimates or an infeasible plan. Completed work is subtracted when replanning through the existing assignment flow. SprintBot directs assignments with completed sessions back to that flow rather than regenerating and discarding progress. An existing approved plan is not replaced merely by previewing a new SprintBot proposal.

3.3 Illustrative example

A student enters a research paper requiring four estimated hours. A writing template proposes research, outlining, drafting, and revision. With four equal steps, each starts at 60 minutes. Suppose the student has two free 120-minute windows before the deadline and no conflicting approved work. The scheduler can allocate four 60-minute sessions. If only one such window exists, the proposal is short by 120 minutes and cannot be approved unchanged.

This example explains the algorithm; it is not a record of a student study or a claim that a research paper always requires four hours. The student is responsible for choosing realistic estimates and checking the rubric.

3.4 Optional separate ChatGPT refinement

After a local proposal, the assistant can prepare a planning prompt containing the assignment and proposed milestones. The student chooses whether to copy it and open ChatGPT. The site does not automatically transmit the assignment, call a model API, or import ChatGPT's answer. Any refined steps must be reviewed and entered by the student.

The integrated Express agent described on the next page uses the same independent scheduler. The separate ChatGPT prompt is an optional alternative, not the implementation of the new tool-calling agent.

4 Express framework and tool-calling AI agent

4.1 Server artifacts and agent loop

`server/app.cjs` implements the Express framework routes, static hosting, request limits, and safe error responses. `server/agent.cjs` implements the Responses API loop. `dist/scheduler.js` is shared by browser and server so model proposals are checked by deterministic code. `render.yaml` declares the Render service, while `package-lock.json` records dependencies for reproducible installation.

The agent has three tools: `read_assignment`, `get_availability`, and `check_schedule`. It must read both context tools before submitting a proposal. The schedule tool checks two to eight milestones, whole-minute estimates, the unchanged effort budget, availability, conflicts, and the deadline. Tool feedback is returned to the model so it can revise an invalid proposal. The loop is limited to six model turns, eight tool calls, and three schedule checks. It cannot approve or save a plan, run arbitrary code, or access other tools.

4.2 Private credentials and consent

The server reads `OPENAI_API_KEY` from Render's private Environment settings, or from an ignored local `.env` file. Only blank variable names are included in `.env.example`. The key is absent from HTML, browser JavaScript, report content, and API responses. Environment files and private keys are excluded by `.gitignore`.

Selecting AI mode requires consent to send the chosen assignment requirements, daily windows, occupied session times, and time zone to the server and OpenAI. Other assignment text is not sent. The provider request uses `store: false`; this setting does not imply zero provider retention. The application should be used with non-sensitive class examples and the institution's data rules.

4.3 Deployment and current verification boundary

Run `npm ci` and `npm start` to serve the framework app. The Render Blueprint uses those commands, a free Node web service, and `/api/health`. `/api/config` reports whether AI is configured without revealing the key. On a GitHub Pages origin, the backend is unavailable and the interface keeps AI mode disabled while preserving local planning.

The Express backend and tool loop have been tested with simulated model responses. **A real-provider request and a Render deployment have not yet been verified.** The owner must connect Render, privately set a valid model key with API credits/access, deploy, and exercise AI mode. The actual Render URL must then be added to this report; no unverified URL is claimed here. The verified public demonstration link remains on page one.

4.4 Operational boundaries

AI requests have payload checks, same-origin browser restrictions, five submissions per visitor per hour, thirty per server per hour, two concurrent runs, and a 90-second timeout. In-memory counters reset on restart and are not a durable billing cap. Provider spending controls and monitoring remain necessary. Visitor-supplied availability is not an authenticated calendar, and future daylight-saving changes may require reviewing session times.

5 Validation, deployment, and reproduction

5.1 Completed verification

Six planner tests passed for the framework update reviewed in this report:

1. Relevant templates for research, coding, exam, and presentation work.
2. Preservation of explicit tasks supplied on separate lines.
3. Distribution of small time estimates without zero-length tasks or lost minutes.
4. Rejection of invalid estimates, missing details, unsupported categories, and past deadlines.
5. Proposals that avoid approved sessions and do not mutate approved work during preview.
6. Reporting of a shortfall when no study time is available.

Six agent tests cover tool use, revision, feasibility, limits, and time zones. Four Express tests cover configuration, hidden files, invalid requests, and key isolation. **All 16 tests passed using simulated provider responses.** Live Render/model verification is pending. Interactive browser testing was unavailable; this is not a full accessibility or security audit.

5.2 Deployment arrangement

The public repository is wrightiv-dotcom/studysprint. GitHub Pages publishes the `docs/` directory on `main`. Branch-based publishing avoids requiring a custom workflow upload. The current deployment is accessible at https://wrightiv-dotcom.github.io/studysprint/.

`DEPLOYMENT.md` contains the Render activation checklist and a five-minute class presentation guide. The framework update and this report are delivered together.

5.3 Reproducing and updating the project

1. Clone the repository, run `npm ci`, then `npm start` for the Express app. The static local-planning edition can still be opened directly from `dist/index.html`.
2. With Node.js available, run `npm test` to execute the planning tests.
3. Edit website files under `dist/`, then run `npm run prepare:pages` to refresh the published copy.
4. Commit the source and published files and push `main`. Confirm the Pages build succeeds before treating an update as live.
5. Compile this report with a LaTeX distribution, for example:
   `tectonic -o output/pdf report/studysprint-report.tex`

Generated LaTeX logs, auxiliary files, credentials, private keys, dependency folders, and local archives do not belong in the public source history. The PDF and its LaTeX source are intentional project deliverables.

5.4 References and project links

- **Live application:** https://wrightiv-dotcom.github.io/studysprint/
- **Source and artifacts:** https://github.com/wrightiv-dotcom/studysprint
- **Deployment history:** https://github.com/wrightiv-dotcom/studysprint/actions
- **AI implementation:** OpenAI function calling; Render Express deployment.
- **GitHub Pages publishing documentation:** Configuring a publishing source for a GitHub Pages site.